Java Day 7

Topics : Abstraction, Abstract class, interface,

Real world Design

Imagine this:

you use:

* ATM machine
* mobile Apps
* Car

you don't know:

* internal code
* wiring
* algorithms

But you still use them easily:

* This is abstraction

1. Abstraction -- Hiding complexity

Show only what is needed, hide how
works.

Mental model

ATM Example.

* you see → withdraw, deposit
* you don't see → database logic,
  encoyption

In java

we use:

* obstract classes
* interfaces

2. ABSTRACT CLASS

A class that is incomplete and cannot be directly used

* Parents knows what should happen but doesn't know how exactly

Ex:-

```
abstract class Animal {
    abstract void sound();

    void sleep() {
        S.O.Pln("Animal sleeps");
    }
}
```

Key points

* Cannot create object of abstract class
* Can have:
    * abstract methods (no body)
    * normal methods (with body)

child class must complete it

```
class Dog extends Animal {
    void Sound() {
        S.O.pln ("Dog barks");
    }
}
```

usage:

```
Animal a = new Dog();
a.sound();
a.sleep();
```

Abstract class = incomplete blueprint

3. INTERFACE – Pure Abstraction

A contract that forces classes to implement methods.

interface = rules you must follow

Ex:

- Driving license rules
- Payment System rules

Eg:

```
interface Payment {
    void pay(double amount);
}
```

implementation:

```
class UPI implements payment {
    public void pay (double amount);
        S.o.pln ("paid via UPI :" +amount
    }
}

class Caad implements Payment {
    public void pay (double amount) {
        S.o.pln("Paid via Caad :" +amoun
    }
}
```

Usage

```
Payment P;
P = new UPI ();
P.pay (100);
P=new Caad ();
P.pay (200);
```

why interface is powerful

* Allows multiple implementations
* Support loose coupling
* Enables scalable systems.

Abstract class vs interface

methods	Abstract + normal	only abstract
variables	Allowed	only constants
multiple inheritance	NO	yes
	partial	full

5. Real understanding

Problem

you are building a payment System

you have

* UPI
* Card
* Net Banking

All do:

pay(amount)

But implementation differs

solution use interface

payment

Now easy to add payment

* no code rewrite
* flexible System